

way. So how does SQL Injection work?

Imagine a web application that constructs a database query using a parameter supplied by the user. Such a scenario is very common; in fact it is the basis of dynamic websites. So if you were looking at your own profile page on a website, in the backend it would be constructing a query such as:

```
SELECT * FROM user_data WHERE user_id = 32
```

One might expect the URL to read something like:

```
http://www.somesocialsite.com/profile.php?userid=32
```

Now depending on how this query is constructed, this may or may not be susceptible to injection. Of course, we will examine the case where it is. An application susceptible to SQL injection, the code of the application would be something like the following:

```
query = "SELECT * FROM user_data WHERE user_id = " +  
getUrlParameter("userid")
```

This language is of course entirely made up, here is how it would be in PHP:

```
$query = "SELECT * FROM user_data WHERE user_id = ".$_  
GET["userid"];
```

A little about PHP to help you understand the example better: The `$_GET[]` array in PHP is an array which holds all the parameters passed to an application via the URL. Similarly the `$_POST[]` array holds all the POST parameters, i.e. the data passed to the page from web forms. In PHP all variables start with a `$` symbol. The `."` operation joins two strings.

Back to the code. What we see here is that the parameter being supplied via the URL is directly being used to construct the SQL query. Now if we were to add SQL code to the parameter "userid" in the request URL, we could get the website to execute that code.

For example, here we could do something like, http://www.somesocialsite.com/profile.php?userid=32;DROP important_table.

Since the parameter we provide will be sent to the SQL interpreter unprocessed, we will be able to run any SQL operation we want on the database! Here for example we are dropping – in SQL that means deleting – the table `important_table`. Not a very creative name, but you get the point! With this small vulnerability, the whole database is lost.

One might of course be running the SQL server with a user account that does not have the privileges for this sort of thing, however there are host of attacks which can still be used. You could for example, use this SQL attack to increase your privileges on the site, or to access the list of password hashes, or email accounts for all users.

Let us take an example from WebGoat. In this example, we will try to modify the salary of the user `jsmith` from 50,000 to 200,000. What we are provided with is a textbox that allows us to look up the salary for any user.